

Canvas Account Log In x assignment 5.4 x Untitled5.ipynb - Colab x +

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwyppgEa-#scrollTo=075cc869

Untitled5.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
age_input = IntText(description="Age:")
email_input = Text(description="Email:")
output_area = Output()

# Create a button to trigger data collection
collect_button = Button(description="Collect Data")

def on_button_click(b):
    with output_area:
        collect_user_data(name_input.value, age_input.value, email_input.value)

collect_button.on_click(on_button_click)

# Display the widgets
display(VBox([name_input, age_input, email_input, collect_button, output_area]))
```

Name: luqman
Age: 019
Email: luqman@gmail.com
Collect Data

Variables Terminal

10:47 PM Python 3

22:50 01-09-2025

Canvas Account Log In x assignment 5.4 x Untitled5.ipynb - Colab x +

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwyppgEa-#scrollTo=075cc869

Untitled5.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
collect_button.on_click(on_button_click)

# Display the widgets
display(VBox([name_input, age_input, email_input, collect_button, output_area]))
```

Name: luqman
Age: 019
Email: luqman@gmail.com
Collect Data

--- Collected Data ---
Name: luqman
Age: 19
Email: luqman@gmail.com

Email Hash (for pseudonymization): a1fdfbace07362c7c6f159ccd03b1310152d38e0ae2dec5e68137059a9228ab4

Variables Terminal

10:47 PM Python 3

22:50 01-09-2025

Canvas Account Log In x assignment 5.4 x Untitled5.ipynb - Colab x +

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwygEa-#scrollTo=075cc869

Guest

Untitled5.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
from ipywidgets import interact, Text, IntText, VBox, Button, Output
from IPython.display import display

def collect_user_data(name, age, email):

    print("\n--- Collected Data ---")
    print(f"Name: {name}")
    print(f"Age: {age}")
    print(f"Email: {email}")
    print("-----")

    import hashlib
    email_hash = hashlib.sha256(email.encode()).hexdigest()
    print(f"Email Hash (for pseudonymization): {email_hash}")

    # Create interactive widgets
    name_input = Text(description="Name:")
    age_input = IntText(description="Age:")
    email_input = Text(description="Email:")
    output_area = Output()
```

WhatsApp

SRU AIML CLUB
~Mahesh: Image

Variables Terminal

22:48
01-09-2025

Canvas Account Log In x assignment 5.4 x Untitled5.ipynb - Colab x +

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwygEa-#scrollTo=00781a50

Guest

Untitled5.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
from textblob import TextBlob

def analyze_sentiment(text):
    """
    Analyzes the sentiment of a given text using TextBlob.

    Args:
        text: The input text string.

    Returns:
        A string indicating the sentiment ('Positive', 'Negative', or 'Neutral').
    """
    if not isinstance(text, str):
        return "Invalid input: Please provide a string."

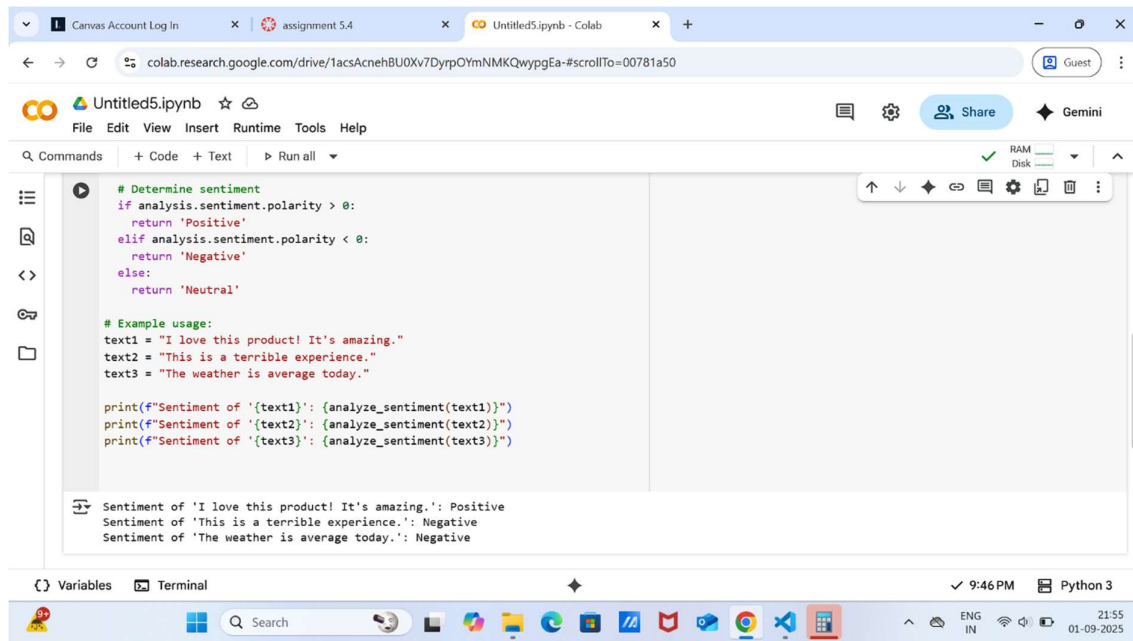
    analysis = TextBlob(text)

    # Determine sentiment
    if analysis.sentiment.polarity > 0:
        return 'Positive'
    elif analysis.sentiment.polarity < 0:
        return 'Negative'
    else:
        return 'Neutral'
```

Variables Terminal

9:46 PM Python 3

21:53
01-09-2025



The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Canvas Account Log In', 'assignment 5.4', and 'Untitled5.ipynb - Colab'. The address bar shows the Colab URL. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The main code area contains a function `analyze_sentiment` that uses `analysis.sentiment.polarity` to determine if text is positive, negative, or neutral. It includes example usage with three text strings. The output cell shows the results for these three texts. The bottom status bar indicates '9:46 PM' and 'Python 3'.

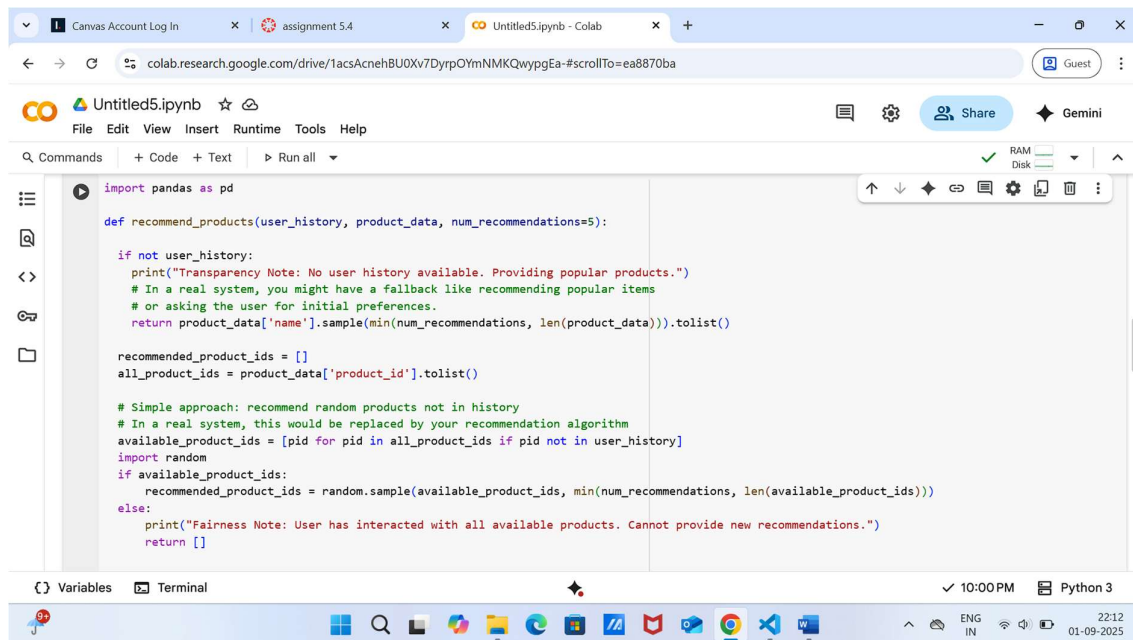
```
# Determine sentiment
def analyze_sentiment(text):
    if analysis.sentiment.polarity > 0:
        return 'Positive'
    elif analysis.sentiment.polarity < 0:
        return 'Negative'
    else:
        return 'Neutral'

# Example usage:
text1 = "I love this product! It's amazing."
text2 = "This is a terrible experience."
text3 = "The weather is average today."

print(f'Sentiment of '{text1}': {analyze_sentiment(text1)}')
print(f'Sentiment of '{text2}': {analyze_sentiment(text2)}')
print(f'Sentiment of '{text3}': {analyze_sentiment(text3)}')
```

Sentiment of 'I love this product! It's amazing.': Positive
Sentiment of 'This is a terrible experience.': Negative
Sentiment of 'The weather is average today.': Negative

Task 3:



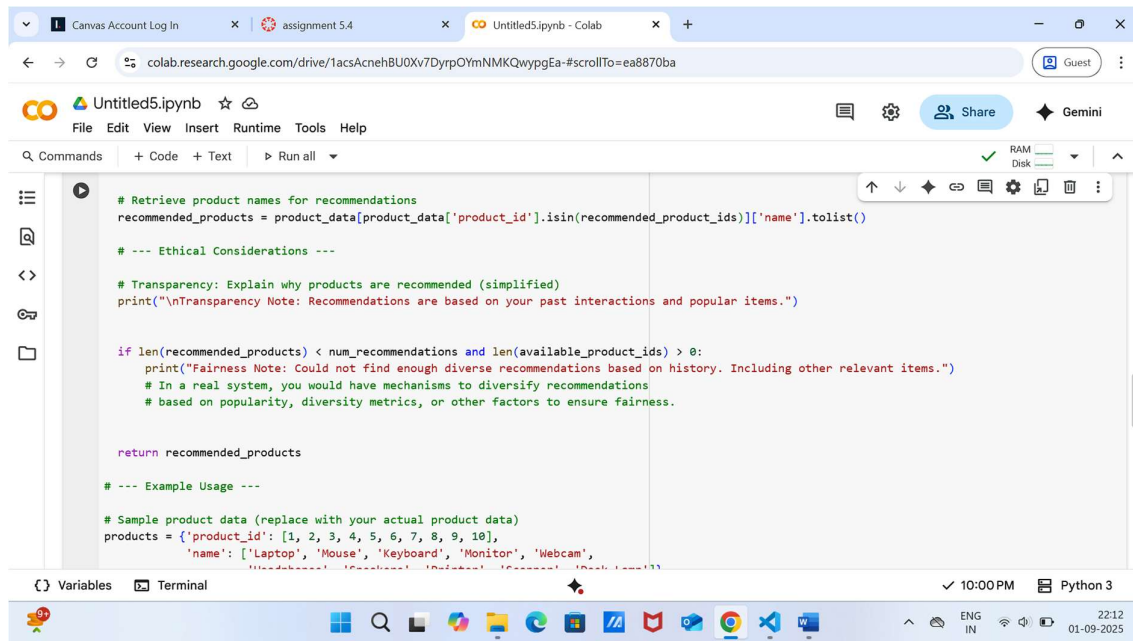
The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Canvas Account Log In', 'assignment 5.4', and 'Untitled5.ipynb - Colab'. The address bar shows the Colab URL. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The main code area contains a function `recommend_products` that takes `user_history`, `product_data`, and `num_recommendations` as arguments. It implements a recommendation algorithm that either samples from popular products if no user history is available or from products not in the user's history. The output cell shows the results for these three texts. The bottom status bar indicates '10:00 PM' and 'Python 3'.

```
import pandas as pd

def recommend_products(user_history, product_data, num_recommendations=5):
    if not user_history:
        print("Transparency Note: No user history available. Providing popular products.")
        # In a real system, you might have a fallback like recommending popular items
        # or asking the user for initial preferences.
        return product_data["name"].sample(min(num_recommendations, len(product_data))).tolist()

    recommended_product_ids = []
    all_product_ids = product_data["product_id"].tolist()

    # Simple approach: recommend random products not in history
    # In a real system, this would be replaced by your recommendation algorithm
    available_product_ids = [pid for pid in all_product_ids if pid not in user_history]
    import random
    if available_product_ids:
        recommended_product_ids = random.sample(available_product_ids, min(num_recommendations, len(available_product_ids)))
    else:
        print("Fairness Note: User has interacted with all available products. Cannot provide new recommendations.")
        return []
```



```
# Retrieve product names for recommendations
recommended_products = product_data[product_data['product_id'].isin(recommended_product_ids)]['name'].tolist()

# --- Ethical Considerations ---

# Transparency: Explain why products are recommended (simplified)
print("\nTransparency Note: Recommendations are based on your past interactions and popular items.")

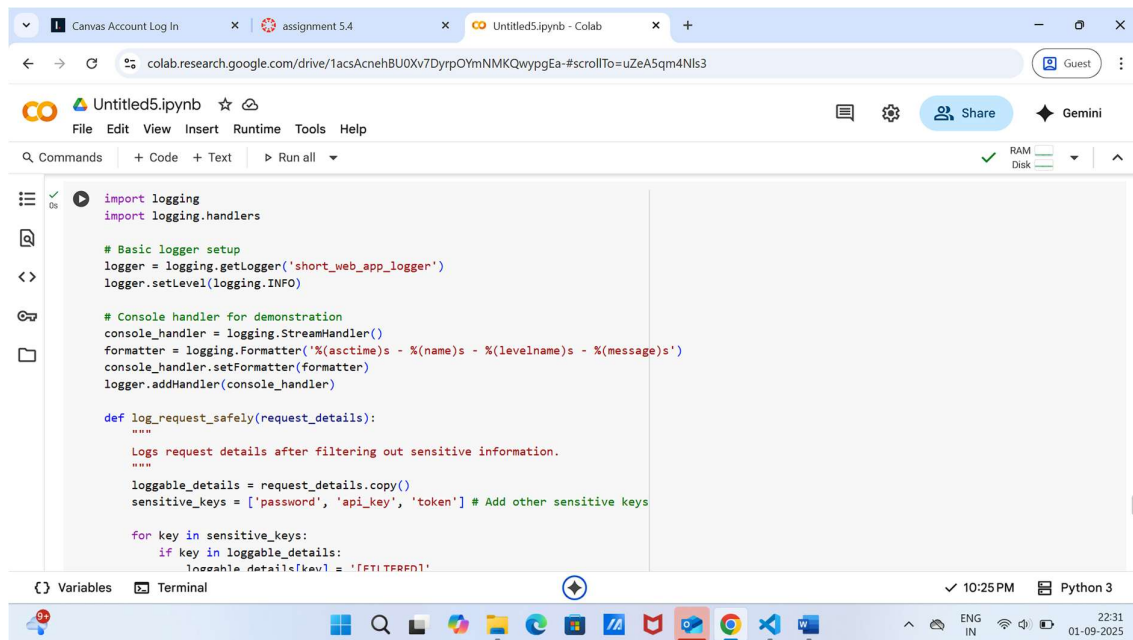
if len(recommended_products) < num_recommendations and len(available_product_ids) > 0:
    print("Fairness Note: Could not find enough diverse recommendations based on history. Including other relevant items.")
    # In a real system, you would have mechanisms to diversify recommendations
    # based on popularity, diversity metrics, or other factors to ensure fairness.

return recommended_products

# --- Example Usage ---

# Sample product data (replace with your actual product data)
products = {'product_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            'name': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Webcam',
                    'Headphones', 'Smartwatch', 'Tablet', 'Smartphone', 'Gaming Console']}
```

Task 4:



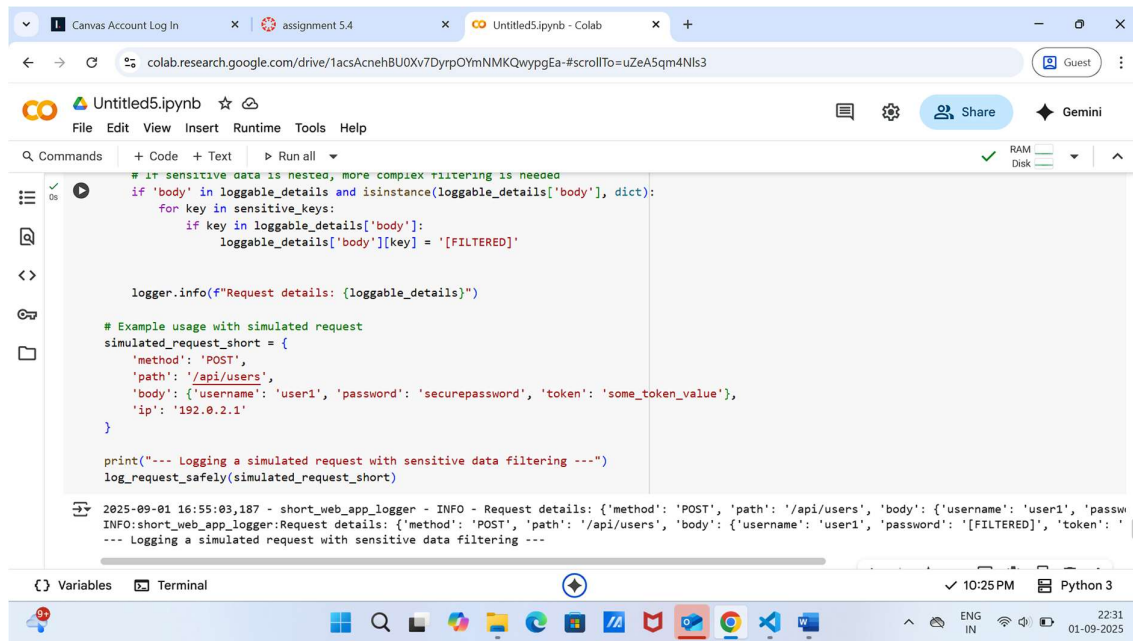
```
import logging
import logging.handlers

# Basic logger setup
logger = logging.getLogger('short_web_app_logger')
logger.setLevel(logging.INFO)

# Console handler for demonstration
console_handler = logging.StreamHandler()
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
console_handler.setFormatter(formatter)
logger.addHandler(console_handler)

def log_request_safely(request_details):
    """
    Logs request details after filtering out sensitive information.
    """
    loggable_details = request_details.copy()
    sensitive_keys = ['password', 'api_key', 'token'] # Add other sensitive keys

    for key in sensitive_keys:
        if key in loggable_details:
            loggable_details[key] = '[REDACTED]'
```



```
# if sensitive data is nested, more complex filtering is needed
def filter_sensitive_data(loggable_details):
    if 'body' in loggable_details and isinstance(loggable_details['body'], dict):
        for key in sensitive_keys:
            if key in loggable_details['body']:
                loggable_details['body'][key] = '[FILTERED]'

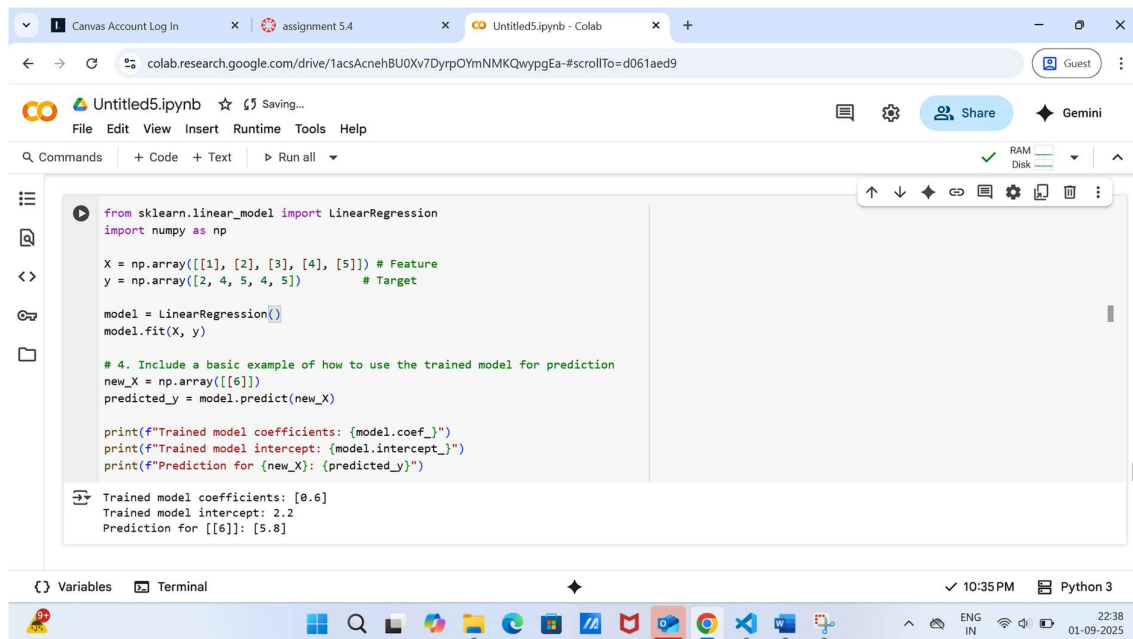
    logger.info(f"Request details: {loggable_details}")

# Example usage with simulated request
simulated_request_short = {
    'method': 'POST',
    'path': '/api/users',
    'body': {'username': 'user1', 'password': 'securepassword', 'token': 'some_token_value'},
    'ip': '192.0.2.1'
}

print("--- Logging a simulated request with sensitive data filtering ---")
log_request_safely(simulated_request_short)

2025-09-01 16:55:03.187 - short_web_app_logger - INFO - Request details: {'method': 'POST', 'path': '/api/users', 'body': {'username': 'user1', 'password': '[FILTERED]', 'token': '[FILTERED]'}, 'ip': '192.0.2.1'}
INFO:short_web_app_logger:Request details: {'method': 'POST', 'path': '/api/users', 'body': {'username': 'user1', 'password': '[FILTERED]', 'token': '[FILTERED]'}, 'ip': '192.0.2.1'}
--- Logging a simulated request with sensitive data filtering ---
```

Task 5:



```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1], [2], [3], [4], [5]]) # Feature
y = np.array([2, 4, 5, 4, 5]) # Target

model = LinearRegression()
model.fit(X, y)

# 4. Include a basic example of how to use the trained model for prediction
new_X = np.array([[6]])
predicted_y = model.predict(new_X)

print(f"Trained model coefficients: {model.coef_}")
print(f"Trained model intercept: {model.intercept_}")
print(f"Prediction for {new_X}: {predicted_y}")

Trained model coefficients: [0.6]
Trained model intercept: 2.2
Prediction for [[6]]: [5.8]
```

Canvas Account Log In | assignment 5.4 | Untitled5.ipynb - Colab

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwyggEa-#scrollTo=6926303a

Untitled5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Before deploying any model, it is crucial to rigorously evaluate its performance on a separate, unseen test data

```
X = np.array([[1], [2], [3], [4], [5]]) # Feature (e.g., number of hours studied)
y = np.array([2, 4, 5, 4, 5]) # Target (e.g., exam score)

model = LinearRegression()

model.fit(X, y)
new_X = np.array([[6]])
predicted_y = model.predict(new_X)

# Print the learned parameters and the prediction
print(f"Trained model coefficients: {model.coef}")
print(f"Trained model intercept: {model.intercept}")
print(f"Prediction for {new_X}: {predicted_y}")
```

Trained model coefficients: [0.6]
Trained model intercept: 2.2
Prediction for [[6]]: [5.8]

Variables Terminal

10:35 PM Python 3

Canvas Account Log In | assignment 5.4 | Untitled5.ipynb - Colab

colab.research.google.com/drive/1acsAcnehBU0Xv7DyrpOYmNMKQwyggEa-#scrollTo=ea8870ba

Untitled5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
# Sample product data (replace with your actual product data)
products = {'product_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            'name': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Webcam',
                    'Headphones', 'Speakers', 'Printer', 'Scanner', 'Desk Lamp']}
product_df = pd.DataFrame(products)

# Sample user history (replace with actual user history data)
user1_history = [1, 3, 5]
user2_history = [2, 4]
user3_history = [] # User with no history

print("Recommendations for User 1:")
recommendations1 = recommend_products(user1_history, product_df)
print(recommendations1)

print("\nRecommendations for User 2:")
recommendations2 = recommend_products(user2_history, product_df)
print(recommendations2)

print("\nRecommendations for User 3 (no history):")
recommendations3 = recommend_products(user3_history, product_df)
print(recommendations3)
```

Recommendations for User 1:

Variables Terminal

10:00 PM Python 3

The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Canvas Account Log In', 'assignment 5.4', and 'Untitled5.ipynb - Colab'. The address bar shows the Colab URL. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The code cell contains the following Python code:

```
print("\nRecommendations for User 3 (no history):")
recommendations3 = recommend_products(user3_history, product_df)
print(recommendations3)
```

The output of the code is displayed in a text area:

```
Recommendations for User 1:

Transparency Note: Recommendations are based on your past interactions and popular items.
['Mouse', 'Headphones', 'Speakers', 'Scanner', 'Desk Lamp']

Recommendations for User 2:

Transparency Note: Recommendations are based on your past interactions and popular items.
['Laptop', 'Webcam', 'Headphones', 'Printer', 'Desk Lamp']

Recommendations for User 3 (no history):
Transparency Note: No user history available. Providing popular products.
['Mouse', 'Printer', 'Speakers', 'Laptop', 'Scanner']
```

At the bottom of the notebook, there are tabs for 'Variables' and 'Terminal'. The system tray at the very bottom shows the time as 10:00 PM, the date as 01-09-2025, and the language as ENG IN.

Task 4: